

EXPERIMENT 13 : Generate a Parse Tree Using CFG

Aim: To generate a parse tree for a given sentence using a Context-Free Grammar in Python.

Program:

```
grammar = {  
    "S": [["NP", "VP"]],  
    "NP": [["Det", "N"]],  
    "VP": [["V", "NP"]],  
    "Det": [["the"], ["a"]],  
    "N": [["student"], ["teacher"], ["book"]],  
    "V": [["reads"], ["likes"]]  
}  
  
def build_tree(symbol, words, index):  
    if symbol not in grammar:  
        if index < len(words) and words[index] == symbol:  
            return (symbol, index + 1)  
        return None  
    for rule in grammar[symbol]:  
        children = []  
        current = index  
        for item in rule:  
            result = build_tree(item, words, current)  
            if result is None:  
                break  
            child, current = result  
            children.append(child)  
    else:  
        return ((symbol, children), current)
```

```

return None

def print_tree(tree, level=0):
    if isinstance(tree, str):
        print(" " * level + tree)
    else:
        symbol, children = tree
        print(" " * level + symbol)
        for child in children:
            print_tree(child, level + 1)

sentence = input("Enter a sentence: ").lower()
words = sentence.split()
result = build_tree("S", words, 0)
if result and result[1] == len(words):
    print("\nParse Tree:")
    print_tree(result[0])
else:
    print("Sentence cannot be parsed.")

```

Output:

The screenshot shows a Python IDE with two panes. The left pane displays the source code for a file named `EXP13.py`, which defines a grammar and a recursive `build_tree` function. The right pane shows the output in the IDLE Shell, where the user has entered the sentence "the student reads a book". The program successfully parses the sentence and displays the following parse tree:

```

S
├── NP
│   ├── Det
│   │   └── the
│   └── N
│       └── student
├── VP
│   └── V
│       └── reads
├── NP
│   ├── Det
│   │   └── a
│   └── N
│       └── book

```